Foundry Local so với llama.cpp: chạy model đơn trên một card (RTX 5070)

Ngày đo 08/11/2026. Chỉ đo hiệu suất, không đo chất lượng câu trả lời. Mọi số liệu là số đo thực.

1. Kết luận

1. **Foundry Local không tốt hơn llama.cpp ở bất kỳ chỉ số hiệu năng nào** trên cùng RTX 5070 và cùng model Qwen3 (4B và 8B). Ngay cả khi bật CUDA, decode chậm hơn 18–19%, prefill chậm hơn 38–46%.
2. **Khác biệt lớn nhất là xử lý nhiều yêu cầu cùng lúc.** Với 4 luồng song song, llama.cpp đạt 530 tok/s (4B) và 367 tok/s (8B); Foundry Local chỉ 136 và 89 tok/s, gần bằng một luồng đơn, tức các yêu cầu được xử lý lần lượt. Đây là cơ chế (không có continuous batching) chứ không phải do tinh chỉnh.
3. **Cấu hình mặc định của Foundry Local chạy GPU qua WebGPU và chậm hơn nhiều:** decode chậm hơn 33–35%, prefill chậm hơn 80–83%. Muốn dùng CUDA phải cài thêm CUDA 12 và cuDNN 9; trong bài đo này cài bằng gói pip.
4. **Không có ứng dụng cho phần lõi của project** (gateway nhiều luồng, ghép GPU qua mạng, model 27–30B). Chỗ phù hợp duy nhất là tầng suy luận cục bộ nhỏ trên máy người dùng, chưa đo ở đây (xem mục 5).

2. Điều kiện đo

	llama.cpp (production)	Foundry Local
Phiên bản	llama-server build 11913 (CUDA)	SDK 2.3.1, ONNX Runtime 1.31, GenAI 0.18
Định dạng model	GGUF Q4_K_M (4B 2,5 GB; 8B 5,0 GB)	ONNX int4 của catalog (CUDA: 4B 2,7 GB, 8B 5,7 GB; WebGPU: 2,9 và 6,1 GB)
Chạy trên GPU qua	CUDA	CUDA 12 + cuDNN 9, hoặc WebGPU (mặc định)
Cấu hình	`-ngl 99 -fa on -c 16384 -np 4`, KV f16	mặc định của dịch vụ web

- Phần cứng: Windows, RTX 5070 12 GB, chỉ một card, không ghép; production được tạm dừng trong lúc đo và đã khôi phục.
- Cùng một client đo cho cả hai (HTTP `/v1/chat/completions` dạng stream, tương thích OpenAI); cùng tokenizer Qwen3 nên số token prompt giống nhau (464 và 1.328 token). Mỗi phép đo lặp 3 lần, lấy trung vị; sinh 192 token. Chạy trên chính máy Windows nên không chịu ảnh hưởng của mạng.

3. Kết quả

Qwen3-4B

Chỉ số	llama.cpp	Foundry Local (CUDA)	Foundry Local (WebGPU, mặc định)
Decode một luồng	**174,8 tok/s**	142,5 tok/s (−19%)	114,0 tok/s (−35%)
Prefill 1.328 token	**8.428 tok/s**	5.191 tok/s (−38%)	1.655 tok/s (−80%)
Độ giật p99 giữa hai token	6,2 ms	7,3 ms	9,2 ms
Thông lượng 4 luồng song song	**530,2 tok/s**	135,7 tok/s (−74%)	109,7 tok/s (−79%)
TTFT tệ nhất khi 4 luồng	51 ms	367 ms	361 ms
Điện năng	**0,93 J/token**	1,09 J/token	1,52 J/token
VRAM đỉnh	5,7 GB	5,9 GB	5,7 GB

Qwen3-8B

Chỉ số	llama.cpp	Foundry Local (CUDA)	Foundry Local (WebGPU, mặc định)
Decode một luồng	**113,5 tok/s**	92,0 tok/s (−19%)	75,6 tok/s (−33%)
Prefill 1.328 token	**5.962 tok/s**	3.248 tok/s (−46%)	995 tok/s (−83%)
Độ giật p99 giữa hai token	9,2 ms	11,1 ms	13,7 ms
Thông lượng 4 luồng song song	**367,2 tok/s**	88,7 tok/s (−76%)	72,9 tok/s (−80%)
TTFT tệ nhất khi 4 luồng	58 ms	500 ms	586 ms
Điện năng	**1,72 J/token**	1,95 J/token	2,59 J/token
VRAM đỉnh	7,8 GB	8,3 GB	8,5 GB

Thời gian nạp model lên GPU: llama.cpp 2,7 s (file đã nằm trong bộ nhớ đệm); Foundry Local 2,5–4,9 s sau khi tải. Lần đầu Foundry Local phải tải model từ catalog: 52–55 s (4B) và 110–115 s (8B).

4. So sánh nguyên nhân

Hiện tượng	Nguyên nhân
Thông lượng 4 luồng của Foundry gần bằng một luồng	Foundry Local thiết kế cho một người dùng tại một thời điểm và không phải server inference (không xếp hàng yêu cầu, không continuous batching), nên 4 yêu cầu chạy lần lượt. llama.cpp gom các luồng vào cùng một lần tính
Prefill chậm hơn 38–46% (CUDA) và 80% (WebGPU)	Đường xử lý prompt của ONNX Runtime GenAI kém tối ưu hơn kernel CUDA của llama.cpp; WebGPU chạy qua lớp đồ họa nên còn chậm hơn [ƯỚC TÍNH, chưa profile]
Decode chậm hơn 19% (CUDA)	Cùng cỡ model và lượng tử 4 bit, nên khác biệt nhiều khả năng đến từ kernel và runtime, không từ kích thước model [ƯỚC TÍNH]
Điện năng mỗi token cao hơn 14–17% (CUDA)	Cùng công suất GPU nhưng sinh ra ít token hơn mỗi giây

5. Giới hạn

- Chỉ đo GPU NVIDIA trên Windows với hai model Qwen3 (4B, 8B). Chưa đo CPU, NPU, Linux hay model lớn hơn; catalog của Foundry Local chỉ có model đến khoảng 20B (gpt-oss-20b, Phi-4, Qwen3-14B), không có Qwen3.8-27B hay Coder-30B.
- Foundry Local dùng ONNX int4 còn llama.cpp dùng GGUF Q4_K_M, nên lượng tử hóa không hoàn toàn giống nhau; chất lượng đầu ra chưa được so sánh.
- Ở llama.cpp, thông lượng 2 luồng (127 và 93 tok/s) thấp hơn một luồng đơn trong cả hai lần đo. Trên RTX 3060 Linux nguyên nhân là hai luồng rơi vào hai slot không kề nhau (xem báo cáo `BAO_CAO_LLAMACPP_THONG_LUONG_2_LUONG`); trên RTX 5070 chưa kiểm tra theo cặp slot. Kết luận không phụ thuộc vào điểm này vì 4 luồng cho kết quả bình thường.
- Mỗi cấu hình chỉ chạy một lượt (3 lần lặp mỗi phép đo). Cài CUDA 12/cuDNN 9 bằng gói pip là một cách thử, không phải cài đặt chính thức của Foundry Local.